BikeNetKit: Developing an open-source bicycle network software suite for urban planners

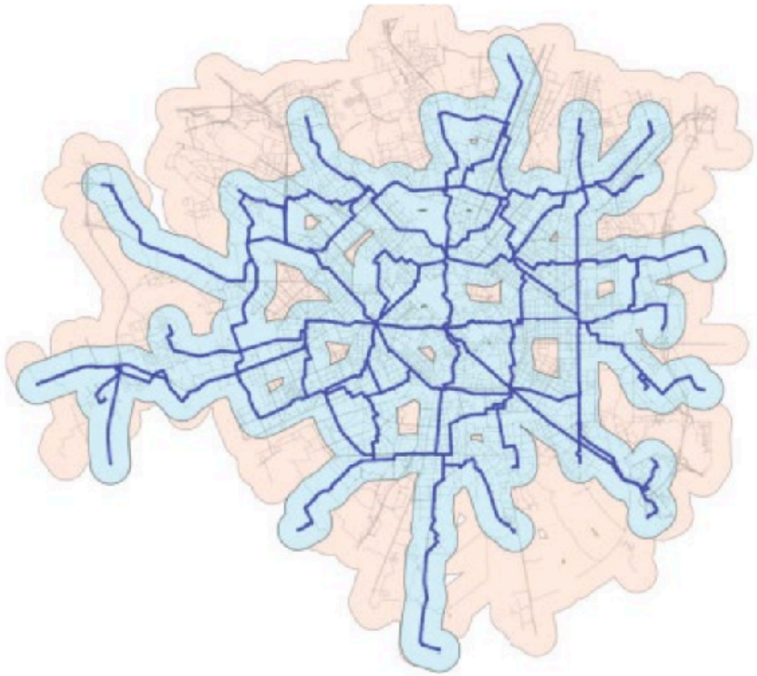
Manuel Knepper
Michael Szell





IT UNIVERSITY OF COPENHAGEN

THE POTENTIAL Cycling is a time-tested benefit to society



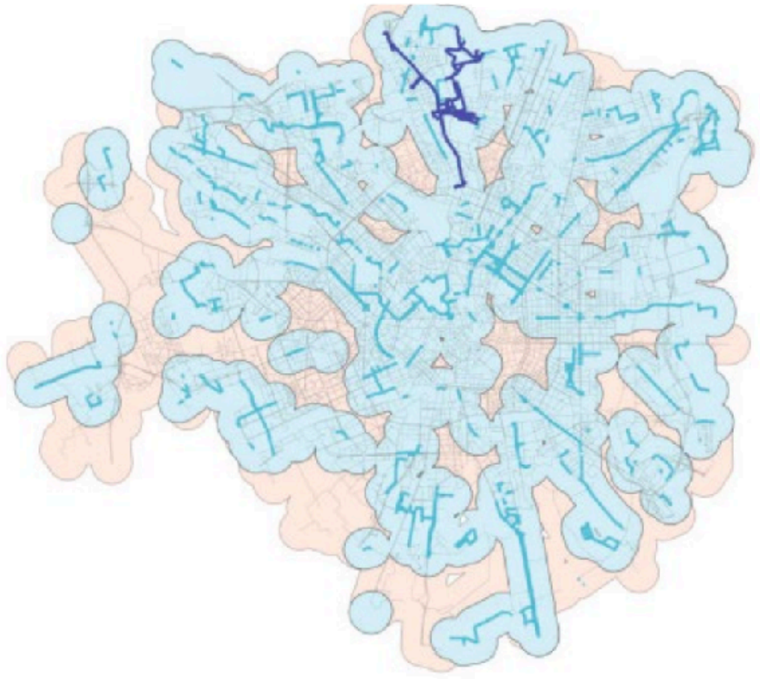
185km

>90 Mio.€

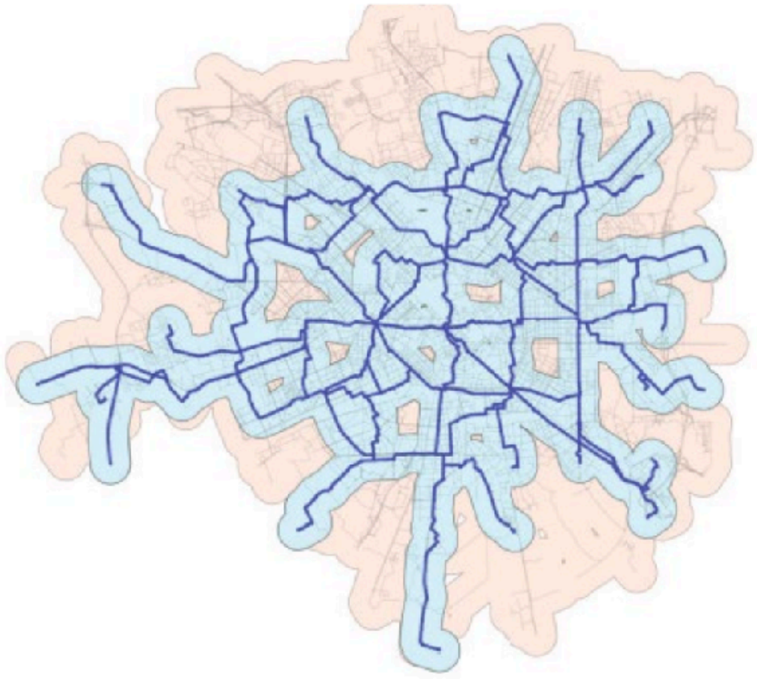
1 km of bike lane:
Breaks even after 3y
Then, +**430000**€ / y!

Szell et al, Sci Rep 12 (2022)
Fosgerau et al, PNAS 120.16 (2023)

THE PROBLEM Cities struggle with bike network planning



Real Milan



Synthetic Milan

185km

>90 Mio.€

Our research shows:
Data-driven planning
produces better networks!

Szell et al, Sci Rep 12 (2022)

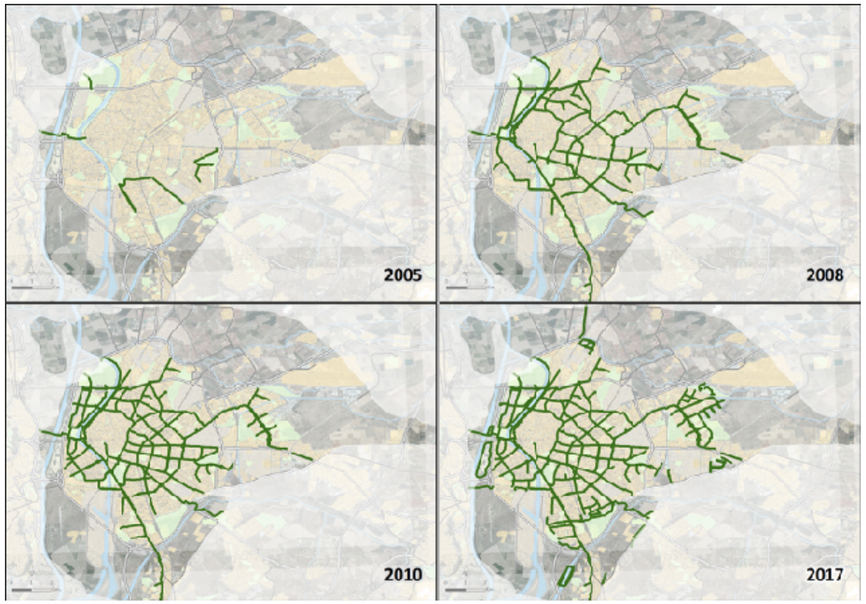
THE PROBLEM Cities struggle with bike network planning

Current network planning is:
- manual
- political
- lacks success metrics

Where to close the gaps?



How to build from scratch?

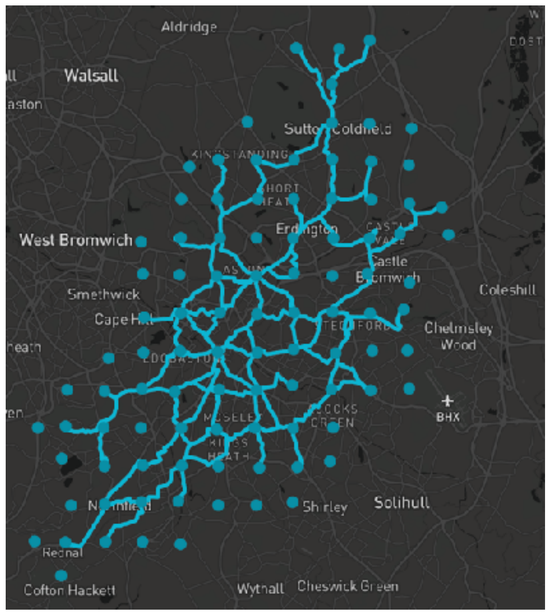


OUR APPROACH Bicycle network algorithms, supporting planners

Here to close the gaps!



This way to build from scratch!



OUR APPROACH Bicycle network algorithms, supporting planners

We have pioneered a ***Science of Bicycle Networks***

Key research/algorithms

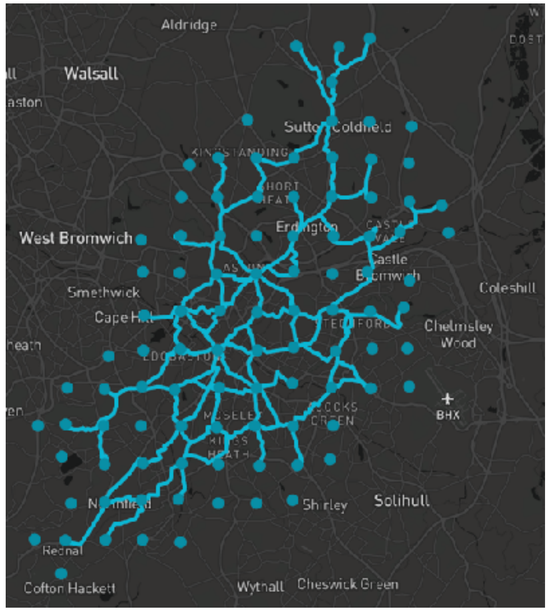
Sebastiao et al, Appl Net Sci (2026)
Szell et al, arxiv:2604.07029 (2026)
Vybornova et al, EPB 52 (2025)
Lonardi et al, R Soc Int 22 (2025)
Vierø et al, EPB 51 (2024)
Büth et al, JOSS 9 (2024)
Folco et al, EPB 50 (2023)
Vybornova et al, Geog Anal 55 (2023)
Szell et al, Sci Rep 12 (2022)
Breum et al, Transp F 56683 (2022)
Natera et al, R Soc Open Sci 7 (2020)

Here to close the gaps!



FixBike.net

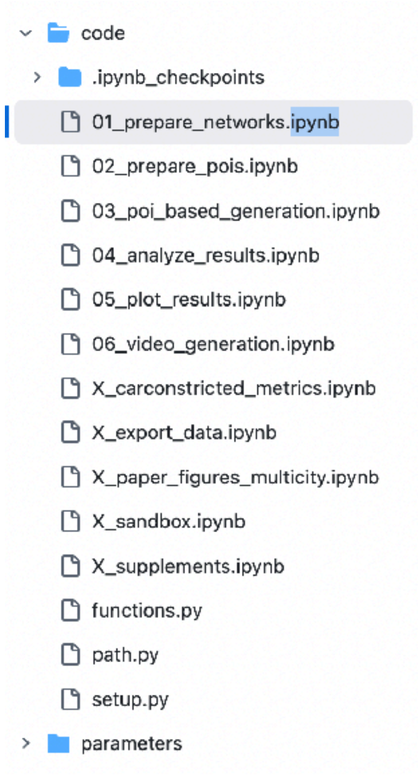
This way to build from scratch!



GrowBike.net

We visualized 2 algorithms, exciting urban planners!

OUR PROBLEM Our algorithms were not user-friendly



Download and wrangle data

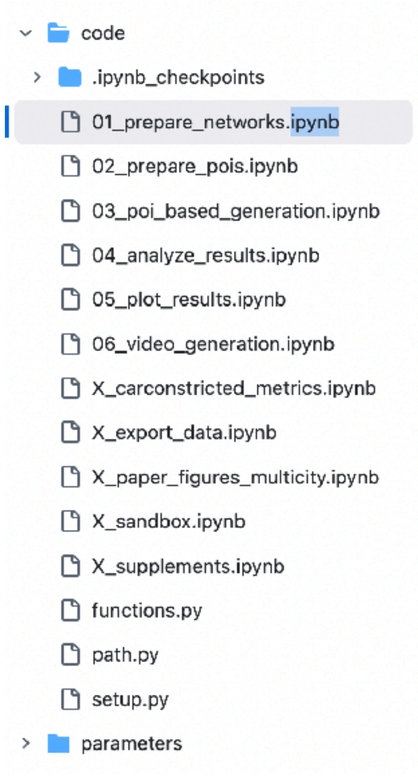
Networks

In [ ]:

```
for placeid, placeinfo in tqdm(cities.items(), desc = "Cities"):
    if placeinfo["nominatimstring"] != '':
        location = ox.geocoder.geocode_to_gdf(placeinfo["nominatimstring"])
        location = fill_holes(extract_relevant_polygon(placeid, shapely.geometry.shape(location['geometry'][0])))
        if debug: # Draw location polygons and their holes
            try:
                color = cm.rainbow(np.linspace(0,1,len(location)))
                for poly,c in zip(location, color):
                    plt.plot(*poly.exterior.xy, c = c)
                    for intr in poly.interiors:
                        plt.plot(*intr.xy, c = "red")
            except:
                plt.plot(*location.exterior.xy)
            plt.show()
    else:
        # https://gis.stackexchange.com/questions/113799/how-to-read-a-shapefile-in-python
        shp = fiona.open(PATH["data"] + placeid + "/" + placeid + ".shp")
        first = next(iter(shp))
        try:
            location = Polygon(shapely.geometry.shape(first['geometry'])) # If shape file is given as linestring
        except:
            location = shapely.geometry.shape(first['geometry'])

    Gs = {}
    for parameterid, parameterinfo in tqdm(osmnxparameters.items(), desc = "Networks", leave = False):
        for i in range(0,10): # retry
            try:
```

OUR PROBLEM Our algorithms were not user-friendly



Download and wrangle data

Networks

In [ ]:

```
for placeid, placeinfo in tqdm(cities.items(), desc = "Cities"):
    if placeinfo["nominatimstring"] != '':
        location = ox.geocoder.geocode_to_gdf(placeinfo["nominatimstring"])
        location = fill_holes(extract_relevant_polygon(placeid, shapely.geometry.shape(location['geometry'][0])))
        if debug: # Draw location polygons and their holes
            try:
                color = cm.rainbow(np.linspace(0,1,len(location)))
                for poly,c in zip(location, color):
                    plt.plot(*poly.exterior.xy, c = c)
                    for intr in poly.interiors:
                        plt.plot(*intr.xy, c = "red")
            except:
                plt.plot(*location.exterior.xy)
            plt.show()
    else:
        # https://gis.stackexchange.com/questions/113799/how-to-read-a-shapefile-in-python
        shp = fiona.open(PATH["data"] + placeid + "/" + placeid + ".shp")
        first = next(iter(shp))
        try:
            location = Polygon(shapely.geometry.shape(first['geometry'])) # If shape file is given as linestring
        except:
            location = shapely.geometry.shape(first['geometry'])

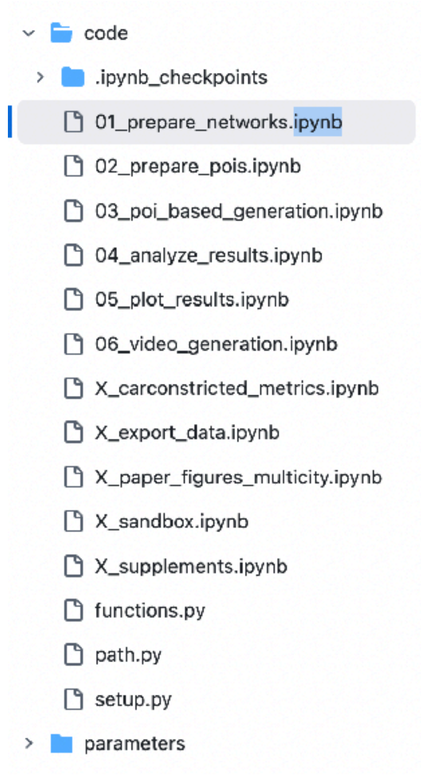
    Gs = {}
    for parameterid, parameterinfo in tqdm(osmnxparameters.items(), desc = "Networks", leave = False):
        for i in range(0,10): # retry
            try:
```

Jupyter notebooks

Spaghetti code



OUR PROBLEM Our algorithms were not user-friendly



Download and wrangle data

Networks

In [ ]:

```
for placeid, placeinfo in tqdm(cities.items(), desc = "Cities"):
    if placeinfo["nominatimstring"] != '':
        location = ox.geocoder.geocode_to_gdf(placeinfo["nominatimstring"])
        location = fill_holes(extract_relevant_polygon(placeid, shapely.geometry.shape(location['geometry'][0]
        if debug: # Draw location polygons and their holes
            try:
                color = cm.rainbow(np.linspace(0,1,len(location)))
                for poly,c in zip(location, color):
                    plt.plot(*poly.exterior.xy, c = c)
                    for intr in poly.interiors:
                        plt.plot(*intr.xy, c = "red")
            except:
                plt.plot(*location.exterior.xy)
            plt.show()
    else:
        # https://gis.stackexchange.com/questions/113799/how-to-read-a-shapefile-in-python
        shp = fiona.open(PATH["data"] + placeid + "/" + placeid + ".shp")
        first = next(iter(shp))
        try:
            location = Polygon(shapely.geometry.shape(first['geometry'])) # If shape file is given
        except:
            location = shapely.geometry.shape(first['geometry'])

    Gs = {}
    for parameterid, parameterinfo in tqdm(osmnxparameters.items(), desc = "Networks", leave = False):
        for i in range(0,10): # retry
            try:
```



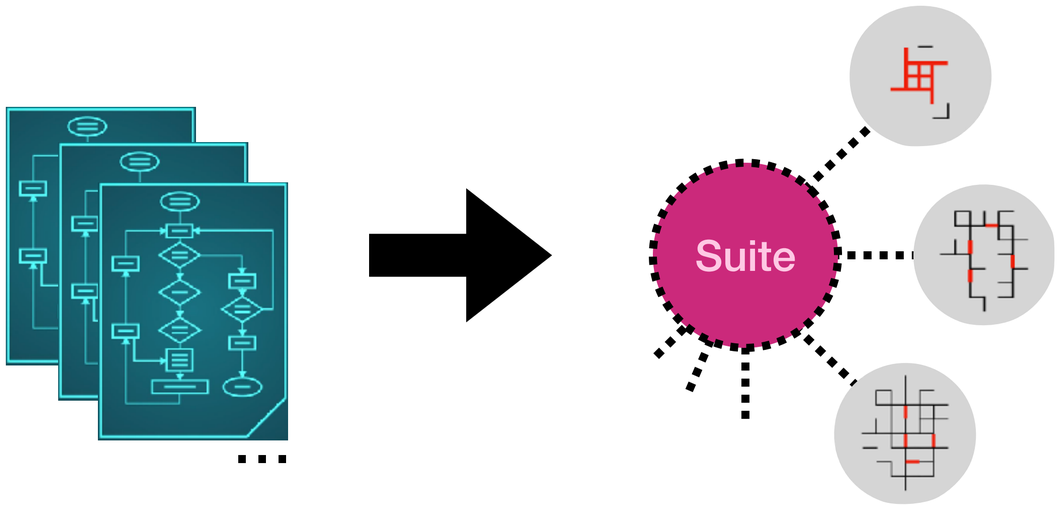
Jupyter notebooks

Spaghetti code



We got funds to 1) Create usable software

Innovation Fund Denmark

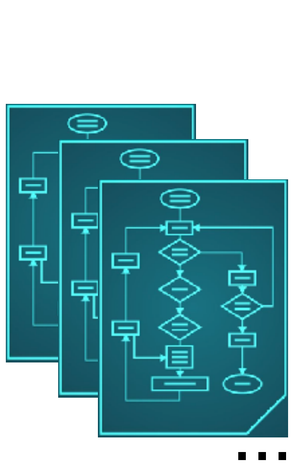


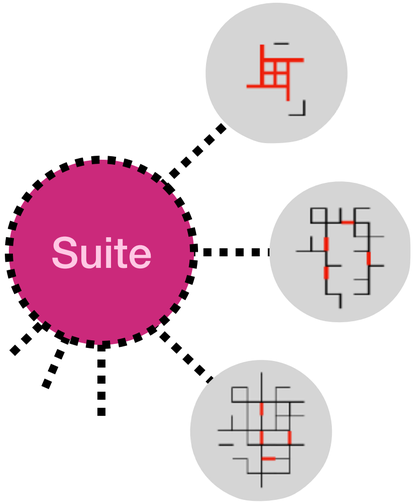
1) Research software engineer (Manuel) creates packaged Python software

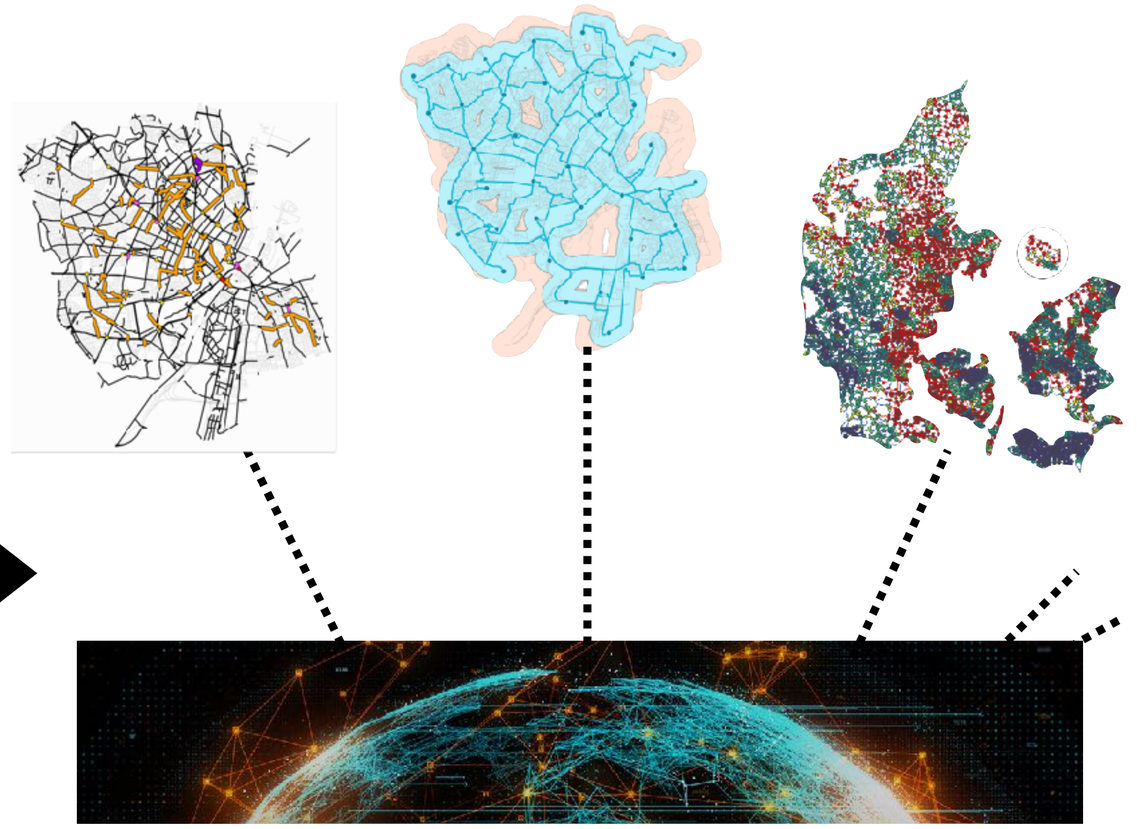
Usable by planners

We got funds to 1) Create usable software, 2) Visualize on the web

Innovation Fund Denmark







1) Research software engineer (Manuel) creates packaged Python software

Usable by planners

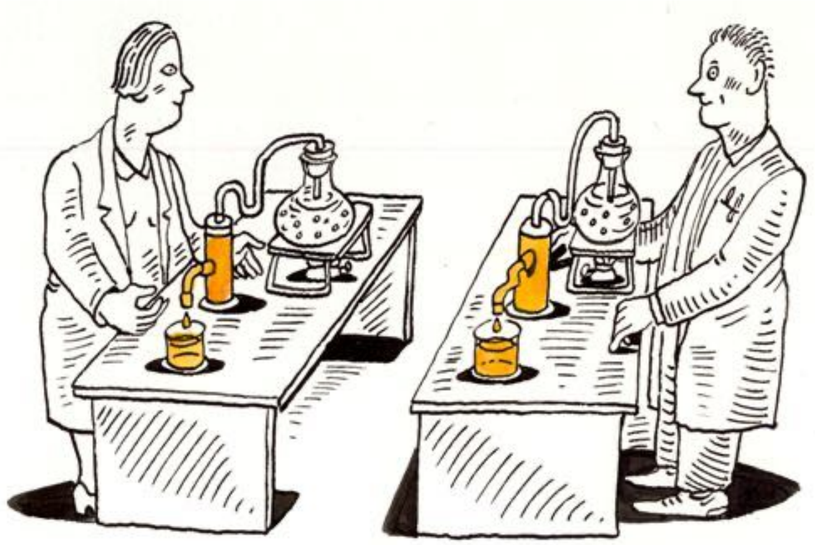
2) Web developer (Moritz) visualizes our tools on a web platform

Guides planners, inspires citizens

Our philosophy: Free, Open-Source, Reproducible

The software should be usable, modifiable, and reproducible by anyone, including proactive citizens, to create a compelling vision for urban cycling in their city.

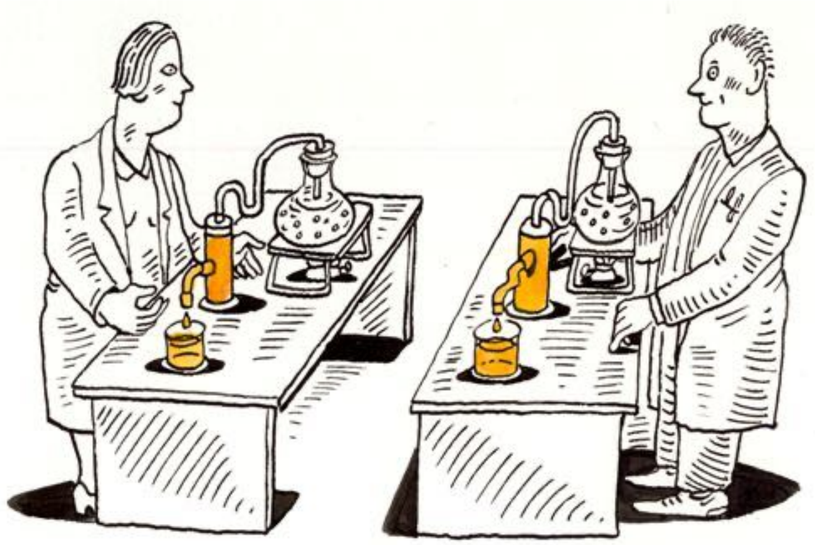




Our philosophy: Free, Open-Source, Reproducible

The software should be usable, modifiable, and reproducible by anyone, including proactive citizens, to create a compelling vision for urban cycling in their city.





It should be an alternative to extractive practices
where cities overpay consultants for a one-off report

AGPL v3
Free Software
Free as in Freedom

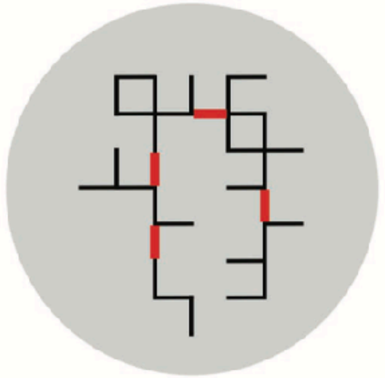
Our tools

We are creating a Python software suite: bikenetkit.org

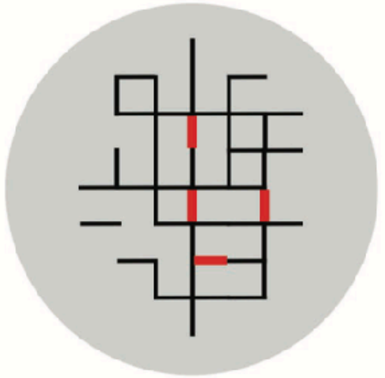
GrowBikeNet



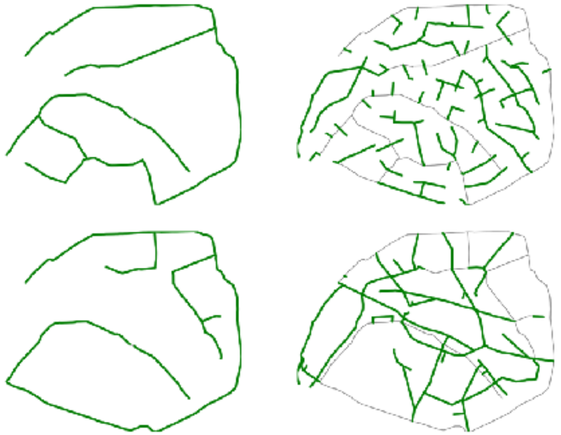
LinkBikeNet



FixBikeNet



SortBikeNet



SUPERBLOCKIFY

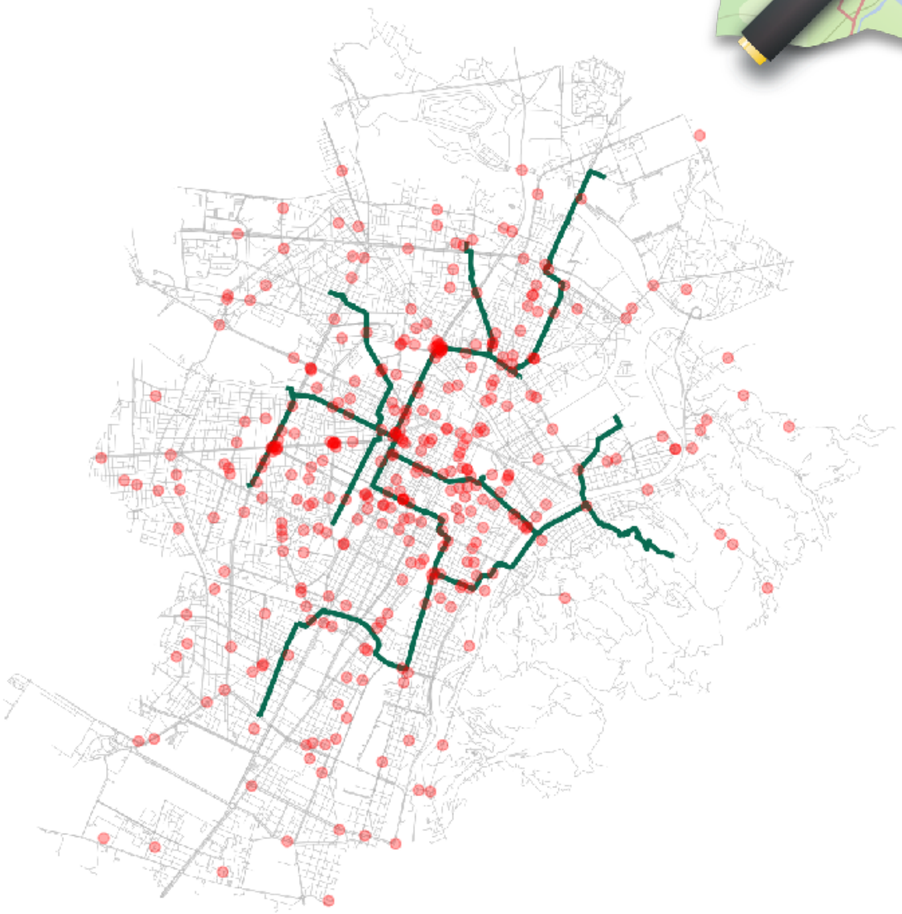
Data



The tools use data from OpenStreetMap

-> open source, high quality

You can import your own data!



How can you help?

- Try out the software. Let us know what works/doesn’t.
- What would you like to improve?
- Follow us / post about it on social media: 
- Star the org/repos on Github: https://github.com/BikeNetKit

All our data and code are open and reproducible



bikenetkit.org



Innovation Fund Denmark

misz@itu.dk

Next: Live demo

Then: Hands-on Tutorial

If you have conda & UNIX/MacOS terminal,
Download & setup now from:

http://tinyurl.com/d3abikenetkit